COMSATS UNIVERSITY ISLAMABAD

BS Business Data Analytics

MACHINE LEARNING LAB PROJECT REPORT

InsureAI

Medical Insurance Cost & Risk Prediction System

Python | scikit-learn | FastAPI | HTML, CSS & JavaScript

SUBMITTED BY

Jaweria Mansoor

ROLL NO.	SECTION	INSTRUCTOR
FA24-BBD-073	5C	Sir Ahmed

PROGRAM

BS Business Data Analytics

UNIVERSITY

COMSATS University Islamabad

Click the table of contents on the next page to navigate this report

Table of Contents

Click any entry to jump straight to it. Each page also carries a **Back to Contents** link in its footer, and the bookmarks panel of your PDF viewer follows the same structure.

Abstract . . . 3

Acknowledgement . . . 3

1. Introduction . . . 3

1.1 Aims of the Project . . . 3

1.2 How the Work Fits Together . . . 4

2. Dataset . . . 4

3. Data Preprocessing . . . 5

4. Machine Learning Models . . . 5

4.1 Linear Regression . . . 6

4.2 Logistic Regression . . . 6

4.3 Decision Tree . . . 6

4.4 Gaussian Naive Bayes . . . 7

4.5 K-Nearest Neighbors (KNN) . . . 7

4.6 Artificial Neural Network (ANN) . . . 7

4.7 Support Vector Machine (SVM) . . . 7

4.8 Random Forest . . . 8

5. Model Evaluation and Results . . . 8

5.1 Results at a Glance . . . 8

5.2 Comparing the Regression Models . . . 8

5.3 What the Results Tell Us . . . 9

5.4 What Drives Insurance Cost . . . 9

5.5 A Short Guide to the Metrics . . . 9

6. FastAPI Backend . . . 9

7. Frontend and Backend Integration . . . 10

8. Technologies and Project Files . . . 10

9. Setup Instructions . . . 11

10. Limitations and Next Steps . . . 11

11. Conclusion . . . 12

Abstract

This report presents **InsureAI**, a machine learning lab project built to estimate medical insurance charges and to sort customers into **High Cost** and **Normal Cost** groups. Working with a single dataset of 1,338 customer records, I applied eight different algorithms to the same problem so that their behaviour could be compared on equal terms, and then connected the trained regression and classification models to a web interface through FastAPI.

Among the results documented in the project record, a pruned Decision Tree gave the best regression performance (R^2 = 0.8641), while Logistic Regression correctly classified 91.04% of the test customers. Smoking status emerged as the single strongest influence on cost, followed by age and BMI. The exact scores of some experiments are missing from the record, and the report says so plainly rather than filling the gaps with estimates.

0.8641	**91.04%**	**1,338**	**8**
Best documented regression R^2 (Pruned Decision Tree)	Classification accuracy (Logistic Regression)	Customer records in the dataset	Machine learning model types explored

Acknowledgement

I would like to thank my instructor, **Sir Ahmed**, for his guidance throughout this machine learning lab, and **COMSATS University Islamabad** for providing the learning environment in which this project was possible. I am also grateful to my classmates in Section 5C for the discussions that helped me think through the problems along the way.

Jaweria Mansoor

BS Business Data Analytics

1. Introduction

Every insurance company faces the same basic question: how much is this customer likely to cost us? The answer shapes how policies are priced and how risk is managed. Traditionally that judgement rests on hand-written rules, but historical records contain far richer patterns than any rule book captures, and machine learning is a natural way to learn from them.

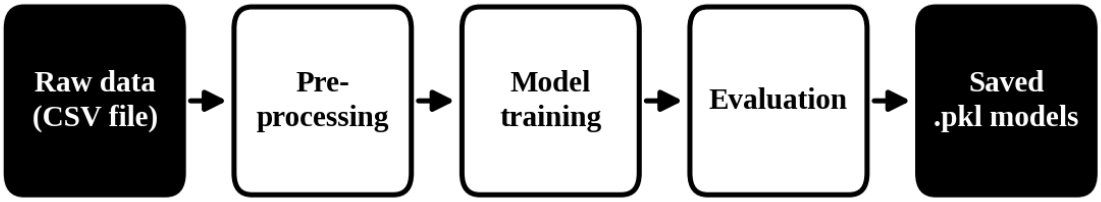
InsureAI explores that idea end to end. It covers data preprocessing, regression, classification, model evaluation and the development of a small web application, using Python, scikit-learn, FastAPI, HTML, CSS and JavaScript.

1.1 Aims of the Project

The project set out to achieve three things. The first was to apply a range of machine learning algorithms to one insurance dataset and compare how they perform. The second was to tackle two related questions: *how much* a customer is likely to be charged (a regression task) and *which cost*

group the customer belongs to (a classification task). The third was to show how trained models can be placed behind a simple web interface so that anyone can use them without touching code.

1.2 How the Work Fits Together



From raw data to deployable models: the workflow followed in InsureAI

Figure 1: The workflow followed in the project, from raw data to saved models.

The rest of this report follows that workflow. It begins with the dataset and its preparation, describes each of the eight models, discusses the results, explains how the models are served through FastAPI, and closes with the limitations that readers should keep in mind.

2. Dataset

The project uses the **Medical Cost Personal Dataset**, stored in *insurance.csv*. It holds **1,338 customer records** across seven columns. Six of them describe the customer and serve as input features; the seventh, *charges*, is what the regression models try to predict.

Column	Role	Type	What it describes
age	Input	Numeric	Age of the customer
sex	Input	Categorical	Gender of the customer
bmi	Input	Numeric	Body mass index
children	Input	Numeric	Number of children or dependants
smoker	Input	Categorical	Whether the customer smokes
region	Input	Categorical	Region where the customer lives
charges	**Target**	Numeric	Medical insurance charge to be predicted

For the classification task, the numeric charge had to be turned into a category. I used the **median charge** (approximately 9,382.033) as the dividing line: customers at or above it are labelled High Cost, and those below it Normal Cost. Splitting at the median keeps the two groups roughly the same size, which makes accuracy a fairer measure. The dataset is small and tidy, which makes it well suited to a lab where the goal is to compare techniques rather than to wrestle with messy data.

3. Data Preprocessing

Machine learning models cannot work directly with raw tables of mixed text and numbers, so the data was examined and prepared first. Different models needed slightly different preparation, but the overall sequence was as follows.

Step	What was done	Why it matters
1	Examined the dataset structure and selected relevant features	Understand the data before modelling it
2	Separated the input variables from the target	Keeps the predictors apart from the answer
3	One-hot encoded sex, smoker and region	Turns categories into numbers a model can use
4	Scaled features where required	Prevents large-valued features from dominating sensitive algorithms
5	Split the data 80/20 with random_state=42	Holds back unseen data for honest testing; repeatable
6	Trained on the training set, evaluated on the test set	Measures how well a model generalises
7	Kept the feature column order consistent	New customer data matches what the model expects



Figure 2: The 80/20 train-test split. Record counts are approximate.

The last step deserves a note. Each trained model is saved together with a list of its feature columns (the *.pkl* files), and the FastAPI application repeats the same encoding on every incoming request. Without this, a missing or re-ordered column could quietly produce wrong predictions at deployment time.

4. Machine Learning Models

Eight model types were used. Some address regression, some classification, and Random Forest was applied to both. The cards below give the overview; each model is then described in turn.

4.1 **Linear Regression** *Regression*	**4.2** **Logistic Regression** *Classification*	**4.3** **Decision Tree** *Regression*	**4.4** **Gaussian Naive Bayes** *Classification*
4.5 **K-Nearest Neighbors** *Classification*	**4.6** **Artificial Neural Network** *Classification*	**4.7** **Support Vector Machine** *Classification*	**4.8** **Random Forest** *Classification + Regression*

***Solid black** = regression* ***Outlined** = classification* ***Heavy outline** = both tasks* *(click a card to jump to its section)*

4.1 Linear Regression

Linear Regression fits a straight-line relationship between the customer's attributes and the insurance charge. It is simple and easy to interpret, which makes it a sensible **baseline** against which more complex models can be judged. Here it reached $\mathbf{R^2 = 0.7836}$ with a mean absolute error of **4,181.19**.

4.2 Logistic Regression

Despite its name, Logistic Regression is a classification method. It estimates the probability that a customer belongs to the High Cost group, using the median-charge rule described earlier.

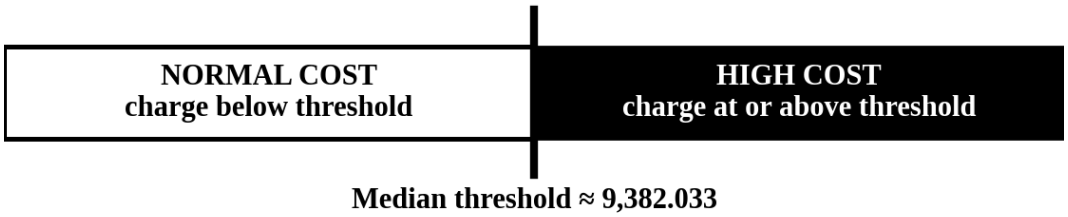
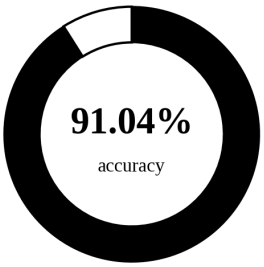


Figure 3: How customers are assigned to the High Cost and Normal Cost classes.



Accuracy = 91.04%

In practical terms, the model labelled about 91 out of every 100 test customers correctly. The web application also reports the underlying probability, so users can see how confident a prediction is rather than receiving a bare label.

4.3 Decision Tree

A Decision Tree predicts charges by asking a series of questions about the customer, such as whether they smoke, and sending them down branches until a prediction is reached. Two versions were built to see what happens when the tree's complexity is controlled: one with no limits and one pruned to a maximum depth of 4 with at least 10 samples in every leaf.

Model	R^2	MAE	RMSE
Decision Tree, no limits	0.6968	3,384.47	6,861.24
Pruned Decision Tree (depth 4, min 10 per leaf)	**0.8641**	**2,697.77**	**4,592.76**

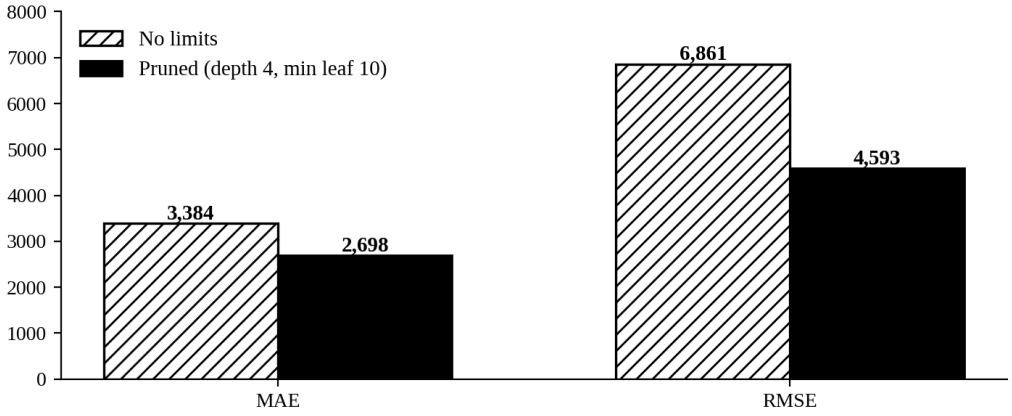


Figure 4: Errors fall noticeably once the tree is pruned (lower is better).

The improvement is a good example of overfitting in action. The unrestricted tree keeps splitting until it has effectively memorised quirks of the training data, which hurts it on new customers. Pruning forces it to learn broader patterns that carry over to data it has not seen.

4.4 Gaussian Naive Bayes

Gaussian Naive Bayes was applied to the classification task. It works with probabilities and makes the simplifying assumption that, once the class is known, the features are independent of one another. That assumption is rarely perfectly true, but it often works surprisingly well and makes the method very fast.

4.5 K-Nearest Neighbors (KNN)

KNN classifies a customer by looking at the customers most similar to them and following the majority. The idea is that people with similar profiles tend to fall into similar cost groups. Because it relies on distances between customers, it is one of the methods for which feature scaling matters.

4.6 Artificial Neural Network (ANN)

The Artificial Neural Network was developed for classification. It is built from layers of interconnected computational units that, together, learn patterns linking the input features to the cost group. It is the most flexible of the classifiers here, though also the hardest to interpret.

4.7 Support Vector Machine (SVM)

SVM was included in the classification experiments, and its data preparation and scaling were worked through in *SVM_Insurance.ipynb*. Its final evaluated score is not part of the available project results, so no SVM performance figure is reported here.

4.8 Random Forest

Random Forest was applied to both classification and regression. Instead of relying on one tree, it builds many and combines their predictions, which usually makes the result steadier than any single tree.

5. Model Evaluation and Results

Classification was judged by **accuracy**, and regression by **R^2, MAE and RMSE** (a short guide to these measures appears at the end of this section). The table collects every model and its documented result.

5.1 Results at a Glance

Model	Task	Result
Linear Regression	Regression	R^2 = 0.7836; MAE = 4,181.19
Logistic Regression	Classification	Accuracy = 91.04%
Decision Tree Regressor	Regression	R^2 = 0.6968; MAE = 3,384.47; RMSE = 6,861.24
Pruned Decision Tree	Regression	**R^2 = 0.8641; MAE = 2,697.77; RMSE = 4,592.76**
Gaussian Naive Bayes	Classification	Evaluated in the lab experiment
KNN	Classification	Evaluated in the lab experiment
ANN	Classification	Evaluated in the lab experiment
SVM	Classification	Exact final score not available
Random Forest	Classification and regression	Evaluated in the lab experiments

A note on missing scores
The exact numbers for Naive Bayes, KNN, ANN and Random Forest are not available in the current project record, so they are not reproduced here. Only documented figures appear in this report.

5.2 Comparing the Regression Models

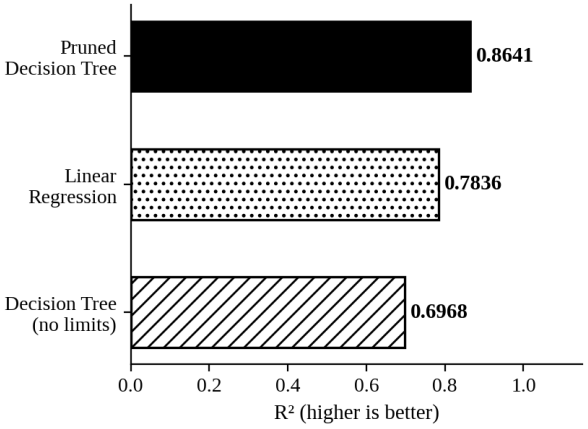


Figure 5: R^2 comparison (higher is better).

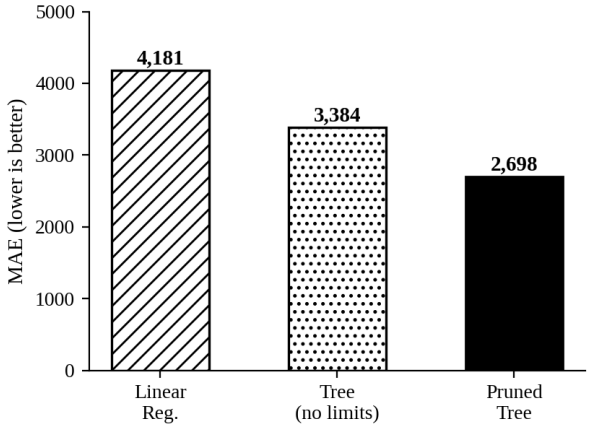


Figure 6: MAE comparison (lower is better).

5.3 What the Results Tell Us

Simpler won. Pruning the Decision Tree raised R^2 by 0.1673, and lowered MAE by 20.3% and RMSE by 33.1%. Against the Linear Regression baseline, the pruned tree cut the average error by about 35.5% (from 4,181.19 to 2,697.77). Among the documented regression results it is clearly the strongest.

One metric is never enough. The unpruned tree actually has a lower MAE (3,384.47) than Linear Regression, yet a lower R^2 (0.6968). Each metric highlights a different side of performance, so a fair comparison needs more than one.

Classification looks strong. Logistic Regression reached 91.04% accuracy on the High Cost versus Normal Cost task. Because accuracy was the only classification measure used, that figure is best read as encouraging rather than conclusive.

5.4 What Drives Insurance Cost

The analysis pointed to a clear ordering of influence:

1	**Smoking status** The strongest factor affecting cost.	**2**	**Age** A major secondary factor.	**3**	**BMI** Also contributes to cost.

5.5 A Short Guide to the Metrics

Metric	In plain words	Better when
R^2	How much of the variation in charges the model explains	Higher (closer to 1)
MAE	The average size of the gap between predicted and actual charges	Lower
RMSE	Similar to MAE, but punishes large mistakes more heavily	Lower
Accuracy	The share of test customers placed in the correct class	Higher

6. FastAPI Backend

Models are only useful if people can reach them, so the next step was a backend. It was written in Python with FastAPI, and its main file, *main.py*, loads the saved regression and classification models, reshapes each incoming customer record into the format those models expect, and sends the prediction back as a response.

Method	Endpoint	Purpose
GET	/	Serves the web application
POST	/predict/regression	Predicts the estimated insurance charge
POST	/predict/classification	Predicts the cost category and its probability
GET	/docs	Opens FastAPI's interactive API documentation

The */docs* page is especially handy during development, because every endpoint can be tested directly in the browser without writing any client code.

7. Frontend and Backend Integration

The web interface is made of three layers working together: **HTML** provides the page structure and input form, **CSS** controls its appearance, and **JavaScript** handles interaction and talks to the backend. The backend serves *index.html*; when a user submits customer details, JavaScript sends them to the prediction endpoints and displays whatever comes back.

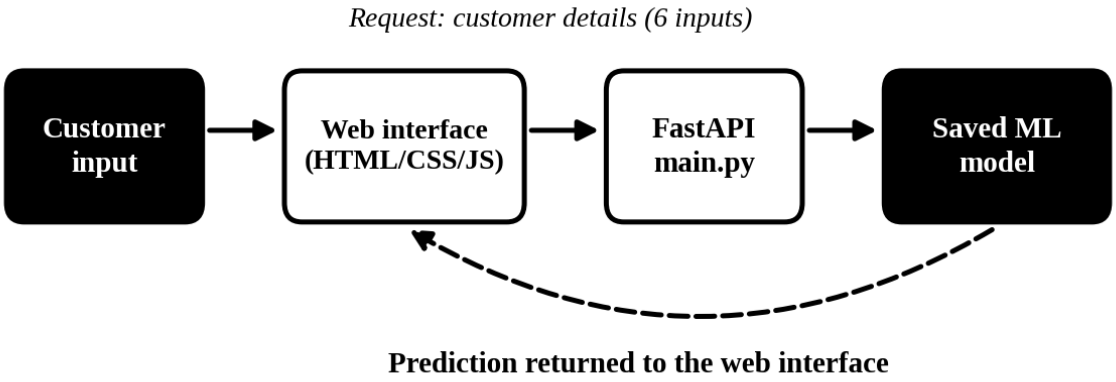


Figure 7: Customer input travels through the web interface and FastAPI to the saved model, and the prediction returns to the page.

> **What the website actually uses**
> The web application uses the **saved regression and classification models** only. The other models built in the lab are separate experiments and are not described as part of the website.

8. Technologies and Project Files

The project relies on a small, widely used toolset: Python for the core work, Pandas for handling data, scikit-learn for building and evaluating models, Joblib for saving them, FastAPI with Uvicorn for the backend, HTML, CSS and JavaScript for the interface, and Jupyter Notebook for the lab experiments.

The files below make up the project. The ones that matter most for running the application are *train_models.py*, *main.py* and *index.html*, together with the saved model files.

File	Description
insurance.csv	Dataset
train_models.py	Trains and saves the models used by the web application
regression_model.py	Regression model script
classification_model.py	Classification model script
Decision_Tree_Insurance.ipynb	Decision Tree experiments
SVM_Insurance.ipynb	SVM preparation
main.py	FastAPI backend

File	Description
index.html	Frontend
regression_model.pkl, classification_model.pkl	Saved models
regression_columns.pkl, classification_columns.pkl	Saved feature-column information
threshold.txt	Classification threshold
requirements.txt	Python dependencies
README.md	Setup instructions

9. Setup Instructions

To run the project, open a terminal in the project directory and install the required libraries, generate the saved model files, and then start the server:

```
pip install -r requirements.txt
python train_models.py
python -m uvicorn main:app --reload
```

Once the server is running, open http://127.0.0.1:8000 in a browser to use the application, or http://127.0.0.1:8000/docs to explore the API. The dataset and the saved *.pkl* files need to be in the correct project directory, so *train_models.py* should always be run first.

10. Limitations and Next Steps

No project is finished, and it is better to be candid about where this one falls short. The table lists the main limitations together with the improvement each one points to.

Limitation	Explanation	Next step
Small dataset, single split	With only 1,338 records, results may shift under a different train/test split.	Use cross-validation.
Limited classification metrics	Classification was judged on accuracy alone.	Add precision, recall, F1-score and a confusion matrix.
Project-defined High Cost label	The threshold is the dataset's median charge, not an official insurance standard.	Treat the label as an academic definition.
Incomplete score record	Scores for Naive Bayes, KNN, ANN, SVM and Random Forest are missing from the available record.	Re-run and record these experiments for a full comparison.

Please note
InsureAI is an academic demonstration. Its predictions are not meant for real insurance pricing or for medical decisions.

11. Conclusion

InsureAI shows how machine learning can be used to estimate medical insurance costs and to group customers by expected cost. Across the lab, eight model types were applied to the same dataset: Linear Regression, Logistic Regression, Decision Tree, Gaussian Naive Bayes, KNN, ANN, SVM and Random Forest.

Three lessons stand out from the documented results. Controlling a model's complexity can matter more than adding to it, as the pruned Decision Tree (R^2 = 0.8641) showed. A single metric rarely tells the whole story. And a good model is only useful once it is served consistently, which is why the saved regression and classification models were placed behind FastAPI and connected to an interactive frontend.

Taken together, the work touches every main stage of a machine learning project, from preparing data to putting a working tool in front of a user. The limitations noted above also give a clear direction for taking it further.

Data preparation	Model development	Evaluation	Model saving	Backend	Frontend

Jaweria Mansoor *| FA24-BBD-073 | Section 5C*
BS Business Data Analytics | Instructor: Sir Ahmed | COMSATS University Islamabad